

SabiID: Proving a Fact Without Revealing the Whole Record

ICSC 2026 Universities Hackathon. Track B, Digital Identity, Trust and ICT. Team Zer0Day Saints.

Live app: <https://sabi-id-icsc.onrender.com/> | Code: <https://github.com/ndujesco/SabiID>

The problem

A shop that sells alcohol needs to know one thing about a customer: whether they are over 18. A bank opening an account needs to know that the name on the form belongs to a verified identity, and that the person standing at the counter is that person. In both cases the question being asked is small, and the honest answer to it is a yes or a no. What actually changes hands, though, is a whole identity document. It gets photographed, saved to a phone, copied into a spreadsheet, forwarded by email. Every one of those copies becomes a place a date of birth, a house address or a NIN can leak from later, and those leaks are exactly what feed the SIM swaps and account takeovers that show up months afterward.

There is a more expensive, public sector version of the same gap. Federal and state payrolls in Nigeria carry ghost workers and ghost pensioners: people who have died, retired without being taken off the books, or never existed at all, kept on the payroll by whoever collects the payment. This happens because once a name is added to a payroll, nobody goes back and asks whether there is still a living, matching person behind it before the next payment is released.

Both problems come down to the same thing. Somebody needs one fact confirmed, and the only tool available to confirm it is the entire record.

What we built

SabiID is a broker that sits between the identity sources and the services that need to check identity. It answers one narrow question at a time, signs the answer, and writes a line to a tamper evident log, without ever copying anyone's record onto its own servers. We deliberately did not attempt to build the national identity gateway our team had originally pitched elsewhere, with its own microservices and a multi year rollout plan. For three weeks we instead carved out the one slice of that idea that is an honest, working prototype, and built it properly.

A mock identity federation stands in for the real sources: one JSON store plays NIMC, holding NIN records, another plays NIBSS, holding BVN records, and a third plays the Civil Registration service, holding deaths. SabiID never copies these records. It reads the single answer it needs at the moment a question is asked, and forgets the underlying record immediately afterward, so there is no master database sitting anywhere for someone to steal. A partner sends a scoped request naming one citizen and a short list of things it needs to know, such as whether the person is verified, whether they are over 18, or whether a name matches. The citizen sees that exact list on their own device and approves it field by field, and anything left unticked simply comes back marked withheld.

Every answer the gateway gives is signed with an Ed25519 key it holds, carrying a fifteen minute expiry and a nonce so a stolen answer cannot be reused or widened later. A partner checks that signature on its own machine, against a public key we publish, with no call back to us required. For the handful of cases where a partner genuinely needs an actual value rather than a yes or no, such as a hospital that needs a name to write on a chart, citizens are issued a credential in the SD-JWT style at enrolment: its signed body holds only salted hashes of each claim, and the plain values live only in the citizen's own wallet. When a value is released, it comes with a short proof tying it back to the signed credential, so nothing beyond that one value is ever disclosed. A face or fingerprint sample used to prove someone is still present and alive is hashed with scrypt the moment it is captured and only that hash is ever stored, so a breach of our database on its own would not hand anyone a usable biometric.

Underneath all of this sits a proof of check log. Every question asked through the gateway becomes one line, and each line carries the cryptographic hash of the line written before it, so the whole log forms a chain. Edit or delete a line after the fact and every hash following it stops matching, and the break points precisely at the line that was tampered with. By construction, a line holds a pseudonymous identifier, the partner asking, the names of the fields requested and the yes or no outcome, and nothing else. It never holds a date of birth, an address or a biometric.

The whole backend runs on the Python standard library with a single dependency, the cryptography package, for the Ed25519 signing. Routing, storage and hashing are all built from what Python ships with, and it will run on an ordinary laptop with nothing else installed.

Why a business would rather do it this way

A business can always simply insist on the full ID, so our design has to be the easier option and not only the more private one. It works out that way for three reasons. A business that holds no copy of a customer's details has nothing to leak and nothing to secure, which is a real reduction in liability. A signed, timestamped attestation that a check happened is also stronger evidence for a regulator than a photocopy sitting in a drawer, and the proof of check log becomes the business's own audit trail for free. And it is faster in practice: there is no form to re key by hand and no document to file away, the answer arrives in the time it takes the citizen to tap approve on their phone. The privacy is genuine, but the reason a shop owner would actually adopt something like this is that it is less work for them, not only a nicer principle.

Ghost workers and ghost pensioners

The payroll case reuses the exact same mechanism, just asked on a schedule instead of once. Before a ministry pays, SabiID sends each line on its roster the same kind of scoped request, asking whether this is still the same living person enrolled against a given employee or pension number. The person answers with a fresh face check on their phone, and the gateway returns a plain yes or no. No raw biometric ever moves and no new record gets created in the process.

When the answer is no, or when nobody answers at all, that one line is held back for manual review, and the corresponding log entry becomes the documented reason the payment was stopped. The audit log stops being a passive privacy feature at that point and starts doing real work for a payroll auditor.

Our synthetic ministry has twelve lines on its roster. Seven are real, active staff who answer correctly each cycle and get paid without incident. Five are held back, and each for a different, honest reason: one has no NIN record at all and was never a real person to begin with, one is listed as deceased by the Civil Registration service and gets caught before we even wait on a response from a phone, one has relocated and simply stopped answering, one offered a sample that does not match the one taken at enrolment, and one is a real member of staff who happened to be out of network signal on the day of the check. On the run we tested, the system releases about 1,379,500 naira in salaries and holds back 955,000 naira across those five lines, with the reason for each hold recorded in the log.

What actually works, and where it still breaks

We tested the parts we could test rather than assume them. The consent flow returns only the fields a citizen approved and nothing else, and we confirmed directly that a date of birth never appears anywhere in a response even when an age question is being answered. Every attestation the gateway issues checks out against its published key, and a result that has been tampered with, or one that has expired, fails that check the way it should. A claim value that was never actually part of a signed credential is rejected outright by the selective disclosure check. Editing a past line in the log is caught precisely at that line, and deleting a line out of the middle of it is caught as a gap. All of this is covered by thirty four assertions in the test suite, run with no external framework.

We would rather be honest about where this still falls short than pretend otherwise. Name matching is exact once case and spacing have been normalised, so a name written in a different order, such as a surname moved to the front, or a dropped accent, reads as a mismatch for the very same person. We left a few such people in our synthetic data on purpose, because that is the real cost of never disclosing the name itself: there is no room left for a human to eyeball a near miss. The living person check has a genuine cost too. A staff member simply out of network signal on the day looks, to the system, identical to a ghost, so it does not guess and instead routes that line to a person for review. In our seeded data that turned out to be one line in twelve. A salary run can be given a grace window of a few missed cycles before anything is withheld, softening this, while a pension run, where there is more to lose from paying a ghost, has no such grace and a single missed cycle is enough.

Our stand in for a biometric sample is a plain string rather than anything device attested, so a stolen enrolment sample in this prototype would defeat the liveness check outright. We salt and hash the enrolment sample so that a database breach on its own would not hand anyone

something usable, but we make no claim here about resisting a deliberate replay attack, and a real deployment would need device attested capture and a fresh challenge on every check to close that gap properly. A hash chain on its own also cannot see its own tail being quietly cut off: if the very last few lines of the log were deleted outright, the shortened chain would still verify cleanly. We added a value the auditor notes down externally to check the log against, which catches this in our prototype, though the clean long term fix is publishing the latest chain hash somewhere outside the system on a schedule. The confidence score we return alongside an answer is a heuristic, not a calibrated probability. It starts at 0.99 and is reduced by fixed amounts when only one identifier could be matched, when the NIN and BVN records disagree on a name, or similar. It is meant as a rough ordering aid for a human reviewer, and we say so plainly next to the number rather than let it look more precise than it is. And the OTP in this prototype is simulated: any six digits are accepted in place of a real one arriving over a citizen's SIM. The consent mechanism itself is real. The second factor behind it, for now, is only a placeholder for one.

Power cuts and dropped networks are ordinary here, so we built with them in mind rather than around them. If the identity sources are unreachable, a payroll cycle pays nobody at all: every line is simply queued, and a single reconcile step replays the queue once the connection returns, with the outage and the recovery each written to the log as their own lines. For citizens on a basic phone with a weak signal, the same approvals work over a USSD short code, the kind already used for mobile banking, with the face check skipped on a feature phone and the citizen told plainly that it was skipped rather than silently ignored.

Beyond the prototype

The parts we left out on purpose, and would build next, are a real second factor with a device attested biometric capture, a zero knowledge age proof sitting behind the same predicate interface we already have, a published checkpoint scheme for the log, and a fuzzy, human in the loop path for a near miss name that still stays inside the consent model rather than working around it. The national rollout, the partner onboarding process and the cross border ambitions from our wider pitch belong to a product roadmap, not a three week prototype, and we have kept them out of this submission deliberately.

Running it

```
python3 -m backend.data_gen      # generate synthetic data, seed the database
python3 -m backend.app           # serve on http://127.0.0.1:8099
python3 tests/run_tests.py       # 34 assertions, no framework
```

A live instance is running at <https://sabi-id-icsc.onrender.com/>, and the source is at <https://github.com/ndujesco/SabiID>.